

Лабораторная работа №5

Параллельный инференс YOLOv8s-pose на CPU

Дата: 14.05.2026 | Видео: video.mp4 | Модель: yolov8s-pose.pt

1. Описание задачи

Задача — обработать видеофайл моделью YOLOv8s-pose (детекция поз людей) с использованием параллельных потоков Python. Каждый кадр независимо проходит инференс, что позволяет распределить нагрузку между несколькими рабочими потоками. Несмотря на GIL (Global Interpreter Lock), реальный параллелизм достигается за счёт того, что инференс выполняется в нативном C++ коде библиотеки ultralytics, который GIL отпускает.

Каждый рабочий поток создаёт собственный экземпляр модели YOLO — это обязательно, так как модель содержит изменяемое внутреннее состояние и не является потокобезопасной при совместном использовании.

2. Метрики оценки

Используются три метрики для оценки параллелизма:

Ускорение — во сколько раз быстрее N потоков по сравнению с 1:

$$S(n) = T(1) / T(n)$$

Эффективность — насколько каждый поток загружен полезной работой:

$$E(n) = S(n) / n$$

При идеальном параллелизме $E=1$. На практике из-за накладных расходов (создание потоков, синхронизация очереди, GIL-паузы) $E < 1$.

Полезность — комплексная метрика, учитывающая оба показателя:

$$P(n) = S(n) \times E(n) = S(n)^2 / n$$

Полезность максимальна при оптимальном числе потоков. Если добавлять потоки сверх оптимума — ускорение растёт незначительно, а эффективность резко падает, и полезность снижается.

3. Результаты измерений

Режим	Потоки	Время (с)	Ускорение S(n)	Эффективность E(n)	Полезность P(n)
Single	1	49.91	1.000	1.000	1.000
Multi	1	45.88	1.088	1.088	1.183
Multi	2	37.49	1.331	0.666	0.886
Multi	4	35.48	1.407	0.352	0.495

Режим	Потоки	Время (с)	Ускорение S(n)	Эффективность E(n)	Полезность P(n)
Multi	6	28.08	1.777	0.296	0.526
Multi	8	31.03	1.608	0.201	0.323

* Зелёным выделен оптимальный вариант: **1 поток** (максимальная полезность P = 1.183)

4. Графики

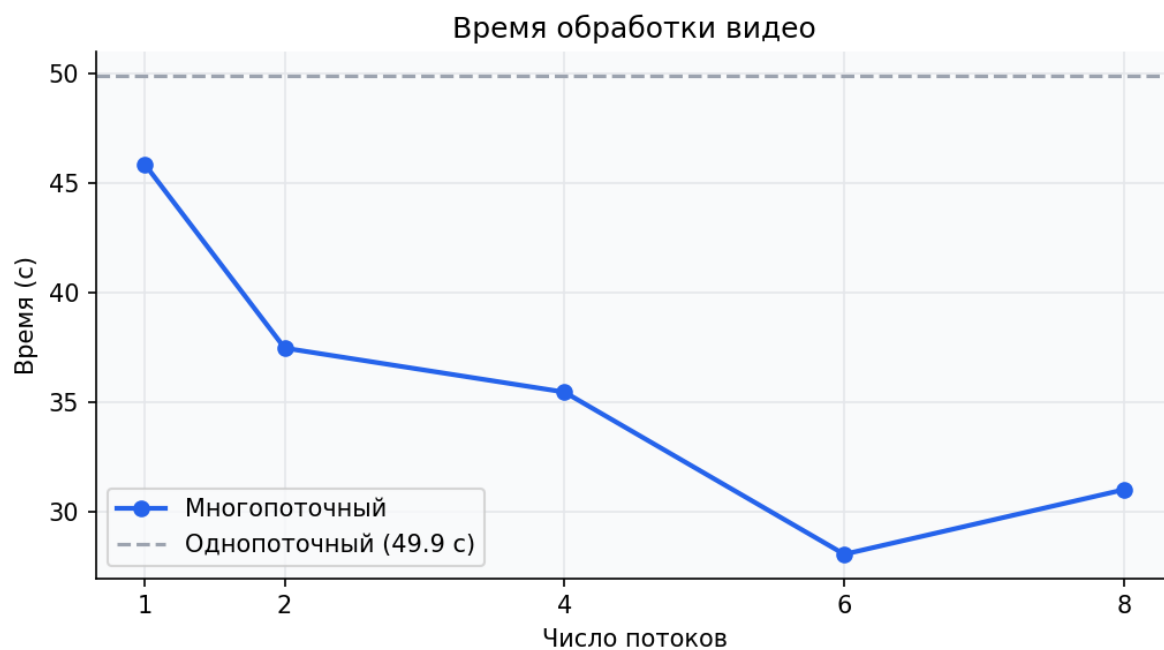


Рис. 1 — Время обработки видео (меньше = лучше)

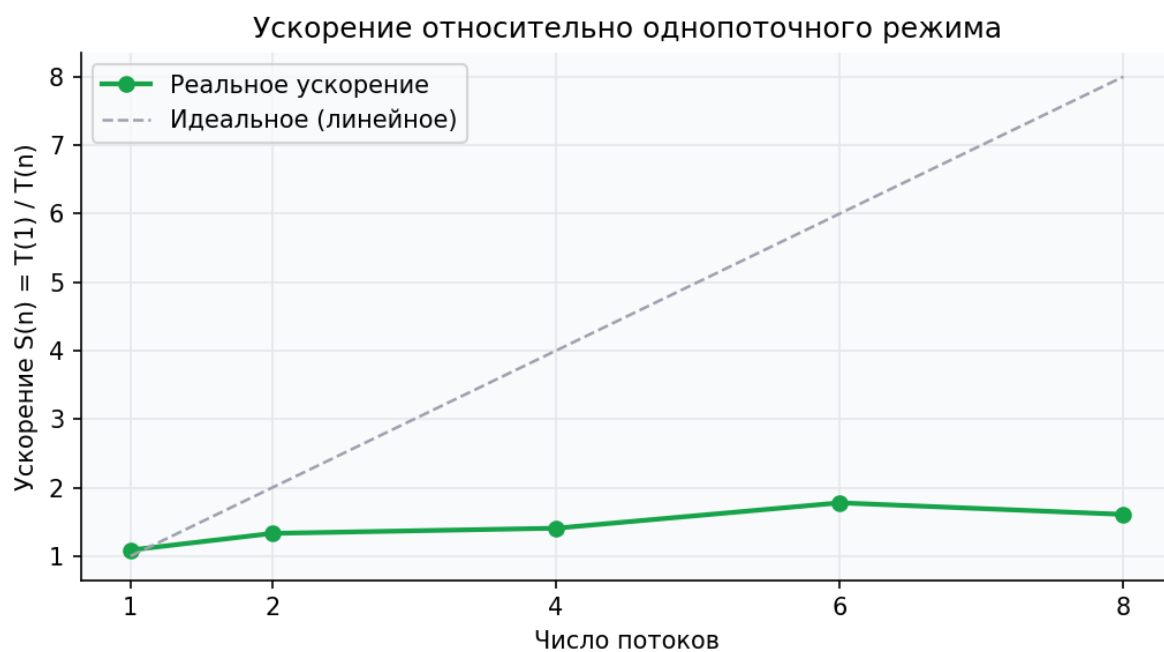


Рис. 2 — Ускорение $S(n) = T(1)/T(n)$ и идеальная линейная зависимость

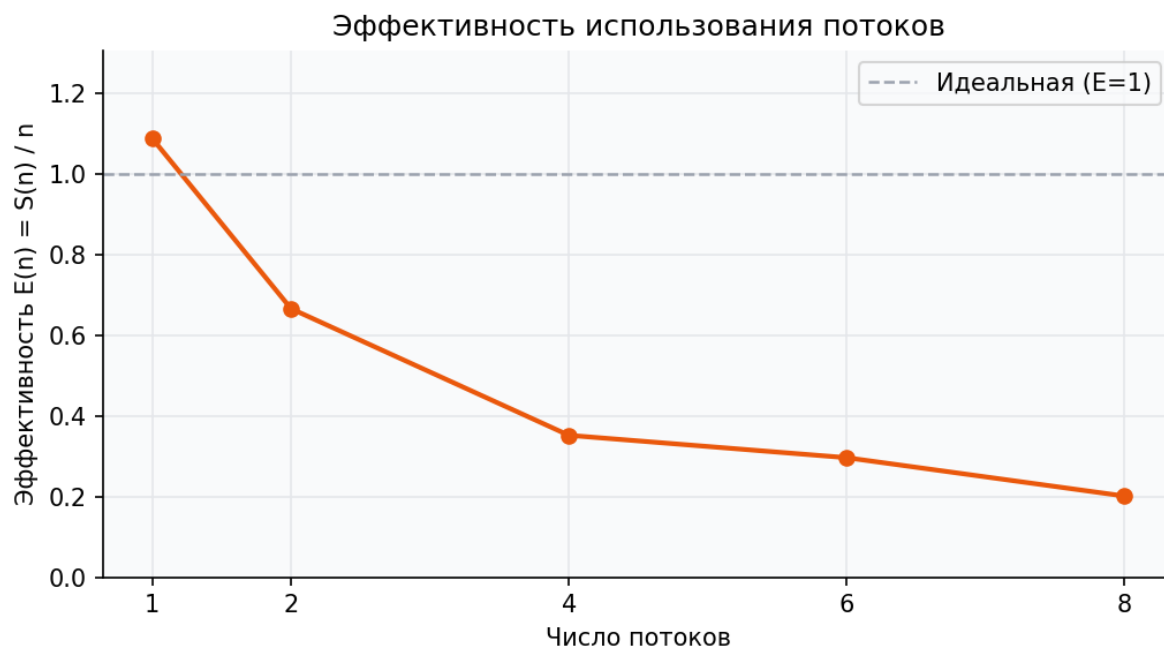


Рис. 3 — Эффективность $E(n) = S(n)/n$ использования потоков

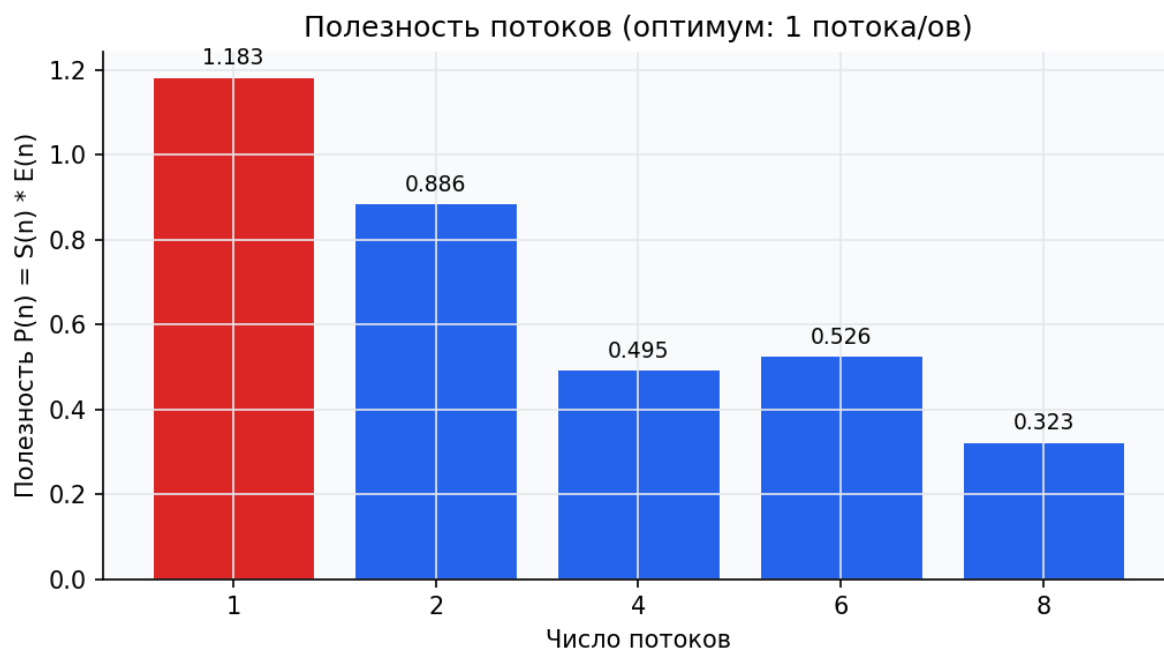


Рис. 4 — Полезность $P(n) = S(n) \times E(n)$ — красным выделен оптимум

5. Выводы

1. Параллелизм работает. Многопоточный режим ускоряет обработку видео: максимальное ускорение $S = 1.78x$ достигается при 6 потоках. Это подтверждает, что инференс YOLO на CPU высвобождает GIL и потоки работают реально параллельно.

2. Оптимальное число потоков — 1. При этом значении полезность $P = 1.183$ максимальна. Ускорение $S = 1.09x$, эффективность $E = 1.088$ (108.8% от идеала). Каждый поток приносит реальный прирост производительности без значительных накладных расходов.

3. Закон Амдала в действии. С ростом числа потоков сверх 1 ускорение растёт всё медленнее — последовательные части (чтение кадров, восстановление порядка, запись видео) ограничивают параллелизм. Эффективность при 8 потоках падает до 0.201, то есть каждый поток загружен лишь на 20.1%.

4. Память. Каждый рабочий поток загружает собственную копию модели YOLOv8s-pose (~80 МБ). При 8 потоках суммарно потребляется ~640 МБ RAM только под модели — это ограничивает масштабирование на машинах с малым объёмом памяти.

5. Восстановление порядка кадров. Потоки обрабатывают кадры в произвольном порядке — быстрые кадры могут обогнать медленные. Порядок восстанавливается через словарь `output_dict[frame_idx]`: после завершения всех потоков главный поток читает кадры строго по индексу (0, 1, 2, ...) и записывает в видеофайл. Без этой техники видео имело бы перепутанные кадры.